

Lecture 4

Key Machine Learning Concepts - Explained
with Linear Regression

Dr. Jomana Bashatah

Slides adapted from Casten Lange

Learning Objectives

- Reviewing the basic idea behind linear regression
- Learning how to measure predictive quality with Mean Square Error (MSE).
- Understanding the role of parameters in a machine learning model in general and in linear regression in particular
- Calculating optimal regression parameters using OLS
- Finding optimal regression parameters by trial and error
- Distinguish between unfitted and fitted models
- Using the `tidymodels` package to split observations from a dataset randomly into a training and testing dataset.
- Understanding how categorical data such as the gender of a person (female/male) can be transformed into numerical dummy variable.
- Being able to distinguish between dummy encoding and one-hot encoding
- Using `tidymodels` including model design and data pre-processing (recipes) to analyze housing prices.

Jumping Right Into It

Univariate OLS With a Real World Example

Data Description

- King County House Sale dataset (Kaggle 2015). House sales prices from May 2014 to May 2015 for King County in Washington State.
- Several predictor variables. For now we use only *Sqft*
- We will only use 100 randomly chosen observations from the total of 21,613 observations.
- We only use *Sqft* as predictor variable for now.

Loading the Data and Assigning Training and Testing Data (Manually)

```
DataHouses=
  import("https://raw.githubusercontent.com/jomanab-cloud/ISE351-25-
Presentations/80f7609c6bcb4887517dcaff62908459cee33e33/HousingData.csv") |>
  clean_names("upper_camel") |>
  select(Price, Sqft=SqftLiving)

# Manually generating DataTrain and DataTest
set.seed(7771)
DataTrain= sample_n(DataHouses, 100)
DataTest= sample_n(DataHouses, 50)
head(DataTrain)
  Price Sqft
1 517000 1180
2 236000 1300
3 490000 2800
4 129000 1150
5 257000 1400
6 312500  870
```

How Much is a House Worth in King County

A house with average properties should be predicted with an average price!

```
MeanSqft=mean(DataTrain$Sqft)
```

```
cat("The mean square footage of a house in King  
county is:", MeanSqft)
```

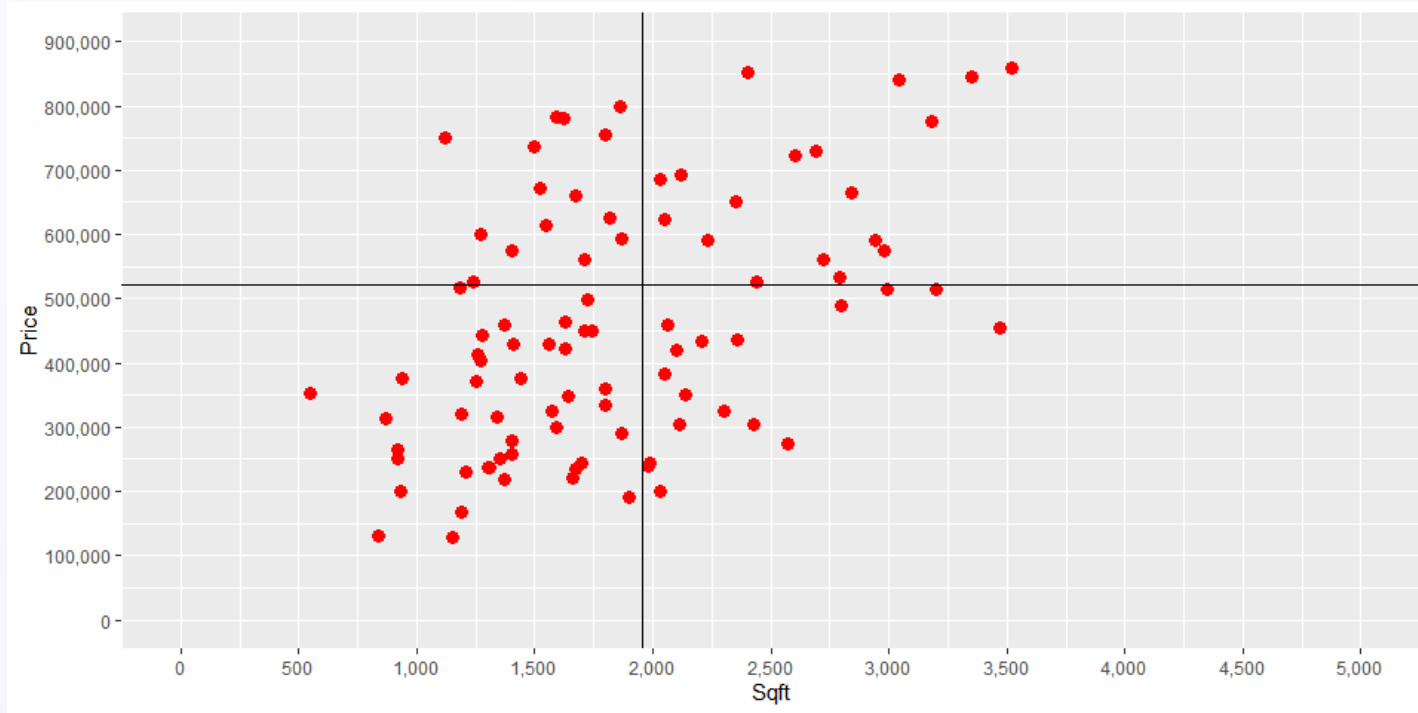
```
The mean square footage of a house in King county  
is: 1956.7
```

```
MeanPrice=mean(DataTrain$Price)
```

```
cat("The mean price of a house in King county  
is:", MeanPrice)
```

```
The mean price of a house in King county is:  
521294.2
```

Predicting the Price of an Average Sized House as the Average of all House Prices



From Unfitted to Fitted Model

How Does the Unfitted Model Look Like

$$\widehat{Price} = \beta_1 Sqft + \beta_0$$

$\underbrace{\quad}_{\hat{y}} \quad \underbrace{\quad}_{\hat{m}} \underbrace{\quad}_{\hat{x}} \quad \underbrace{\quad}_{\hat{b}}$

Fitting the Model with Tidymodels

```
# A tibble: 2 × 5
```

	term	estimate	std.error	statistic	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	(Intercept)	52509.	64183.	0.818	4.15e- 1
2	Sqft	240.	30.6	7.84	5.67e-12

Unfitted Model VS Fitted Workflow Model

Unfitted Model:

$$\widehat{Price} = \beta_1 Sqft + \beta_0$$

$\tilde{y}$ $\tilde{m}$ $\tilde{x}$ $\tilde{b}$

A tibble: 2 × 5

term	estimate	std.error	statistic	p.value
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	52509.	64183.	0.818	4.15e- 1
2 Sqft	240.	30.6	7.84	5.67e-12

Fitted Model:

$$\widehat{Price} = 240Sqft + 52509$$

$\tilde{y}$ $\tilde{m}$ $\tilde{x}$ $\tilde{b}$

Interpretation and Significance

```
# A tibble: 2 × 5
```

	term	estimate	std.error	statistic	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	(Intercept)	52509.	64183.	0.818	4.15e- 1
2	Sqft	240.	30.6	7.84	5.67e-12

$$\widehat{Price} = 240 \cdot Sqft + 52509$$

$$(+240) = 240 \cdot (+1) + (+0)$$

$$(+480) = 240 \cdot (+2) + (+0)$$

$$(+720) = 240 \cdot (+3) + (+0)$$

For each extra *Sqft* the predicted price increases by \$240

The variable *Sqft* is significant. I.e., the probability that the related coefficient β_1 equals zero is extremely small.

Evaluating Predictive Quality with the Training and Testing Dataset

```
DataTrainWithPred = augment(WFModelHouses, new_data=DataTrain)
metrics(DataTrainWithPred, truth=Price, estimate = .pred)
```

```
# A tibble: 3 × 3
```

	.metric	.estimator	.estimate
	<chr>	<chr>	<dbl>
1	rmse	standard	230129.
2	rsq	standard	0.385
3	mae	standard	174350.

```
DataTestWithPred=augment(WFModelHouses, new_data=DataTest)
metrics(DataTestWithPred, truth=Price, estimate=.pred)
```

```
# A tibble: 3 × 3
```

	.metric	.estimator	.estimate
	<chr>	<chr>	<dbl>
1	rmse	standard	163476.
2	rsq	standard	0.626
3	mae	standard	132050.

Univariate Linear Regression - Data Table and Goal

The Regression:

$$\hat{y}_i = \beta_1 x_i + \beta_2$$

The Goal

Find values for β_1 and β_2 that minimize the prediction errors $(y_i - \hat{y}_i)^2$

**The Data Table
Mockup Training Dataset**

i	yGrade	xStudyTime
1	65	2
2	82	3
3	93	7
4	93	8
5	83	4

Univariate Linear Regression - Data Diagram and Goal

The Data Diagram

The Regression:

$$\widehat{Grade}_i = \beta_1 \cdot StudyTime_i + \beta_2$$

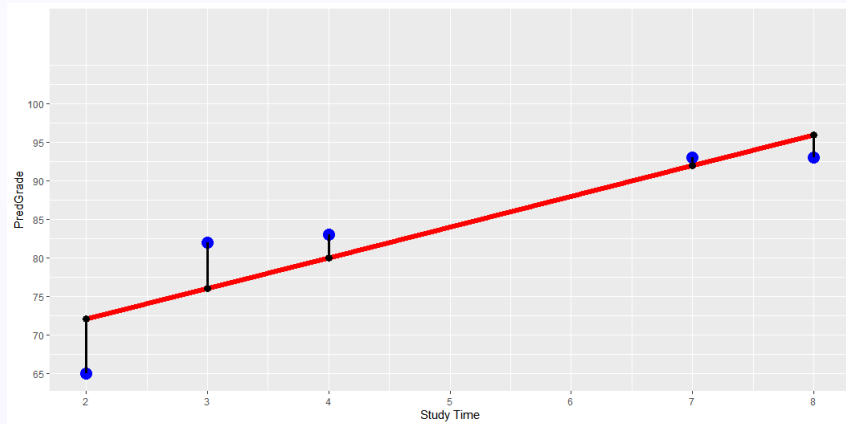
The Goal

Find values for β_1 and β_2 that minimize the prediction errors

$$(y_i - \hat{y}_i)^2$$

The Optimal Prediction

$$\widehat{Grade}_i = 4 \cdot StudyTime_i + 64$$



How to Measure Prediction Quality

$$MSE = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

$\Leftrightarrow$

Prediction

$$MSE = \frac{1}{N} \sum_{i=1}^N (\overbrace{\beta_1 x_i + \beta_2}^i - y_i)^2$$

Error
 i

Note: when the data are given (i.e., x_i and y_i are given), the MSE depends only on the choice of β_1 and β_2

How to Measure Prediction Quality with the MSE

$$MSE = \frac{(\beta_1 x_1 + \beta_2 - y_1)^2 + (\beta_1 x_2 + \beta_2 - y_2)^2 + \dots + (\beta_1 x_5 + \beta_2 - y_5)^2}{5}$$

$\Leftrightarrow$

$$MSE = \frac{1}{5} \left[\begin{array}{cc} \text{Prediction 1} & \text{Prediction 2} \\ \underbrace{(\beta_1 \cdot 2 + \beta_2)}_{\text{Error 1}} - 65 & \underbrace{(\beta_1 \cdot 3 + \beta_2)}_{\text{Error 2}} - 82 \end{array} \right]^2 +$$

$$+ \left[\begin{array}{cc} \text{Prediction 3} & \text{Prediction 4} \\ \underbrace{(\beta_1 \cdot 7 + \beta_2)}_{\text{Error 3}} - 93 & \underbrace{(\beta_1 \cdot 8 + \beta_2)}_{\text{Error 4}} - 93 \end{array} \right]^2 +$$

$$+ \left[\begin{array}{c} \text{Prediction 5} \\ \underbrace{(\beta_1 \cdot 4 + \beta_2)}_{\text{Error 5}} - 83 \end{array} \right]^2$$

Custom R Function to Calculate MSE

Function Call:

```
VecBetaTest=c(4,61)
ResultMSE=FctMSE(VecBetaTest, DataMockup)
print(ResultMSE)
[1] 29.8
```

Function Definition:

```
FctMSE=function(VecBeta, data){
  Beta1=VecBeta[1]
  Beta2=VecBeta[2]
  data=data |>
  rename(Y=1, X=2) |>
  mutate(YPred=Beta1*X+Beta2) |>
  mutate(Error=YPred-Y) |>
  mutate(ErrorSq=Error^2)

  MSE=mean(data$ErrorSq)

  return(MSE) }
```

How to Find Optimal Values for β_1 AND β_2

Method 1:

Calculate optimal values for the parameters (the β s) based on Ordinary Least Squares (OLS) using two formulas (**Note**, this method works only for linear regression)

Method 2:

We can use a **systematic trial and error process**.

Method 1: Calculate Optimal Parameters (Only for OLS)

$$\beta_{1,opt} = \frac{N \sum_{i=1}^N y_i x_i - \sum_{i=1}^N y_i \sum_{i=1}^N x_i}{N \sum_{i=1}^N x_i^2 - (\sum_{i=1}^N x_i)^2} = 3.96$$
$$\beta_{2,opt} = \frac{\sum_{i=1}^N y_i - \beta_1 \sum_{i=1}^N x_i}{N} = 64.18$$

i	Grade	StudyTime	GradeXStudyTime	StudyTimeSquared
1	65	2	130	4
2	82	3	246	9
3	93	7	651	49
4	93	8	744	64
5	83	4	332	16

Mockup Training Dataset

Grade	StudyTime	GradeXStudyTime	StudyTimeSquared
416	24	2103	142

Column Sums

Method 2: Use a Systematic Trial and Error Process

- Grid Search (aka Brute Force):
 1. For a given range of β_1 and β_2 values, build a table with pairs of all combinations of these β_s
 2. Then use our custom `FctMSE()` command to calculate a *MSE* for each β pair.
 3. Find the β pair with the lowest *MSE*
- **Optimizer:** Use the R build-in optimizer. Push the start values for β_1 and β_2 together with the data to the optimizer as arguments. The rest is done by the optimizer.

Manual Trial and Error (AKA Brute Force)

$$\widehat{Grade}_i = \beta_1 \cdot StudyTime_i + \beta_2$$

```
GuessBeta1=0
GuessBeta2=83
MSE=FctMSE(c(GuessBeta1,GuessBeta2), data=DataMockup)
cat("The MSE for beta1=",GuessBeta1, "and
beta2=",GuessBeta2, "is:", MSE)
# Static fallback for non-HTML formats (pptx): {webr} live
cells only run in the browser
GuessBeta1=0
GuessBeta2=83
MSE=FctMSE(c(GuessBeta1,GuessBeta2), data=DataMockup)
cat("The MSE for beta1=",GuessBeta1, "and
beta2=",GuessBeta2, "is:", MSE)
The MSE for beta1= 0 and beta2= 83 is: 105
```

More Systematic Manual Trial and Error (still Brute Force)

Average StudyTime leads to a prediction of average grade:

$$\overline{StudyTime} = \text{mean}(StudyTime) = 4.8$$

$$\overline{Grade} = \text{mean}(Grade) = 83.2$$

Therefore:

$$\overline{Grade} = \beta_1 \cdot \overline{StudyTime} + \beta_2$$

$$\beta_2 = \overline{Grade} - \beta_1 \cdot \overline{StudyTime}$$

```
GuessBeta1=0
```

```
GuessBeta2=83.2 - GuessBeta1 * 4.8
```

```
MSE=FctMSE(c(GuessBeta1,GuessBeta2), data=DataMockup)
```

```
cat("The MSE for beta1=",GuessBeta1, "and beta2=",GuessBeta2, "is:", MSE)
```

```
# Static fallback for non-HTML formats (pptx): {webr} live cells only run in the browser
```

```
GuessBeta1=0
```

```
GuessBeta2=83.2 - GuessBeta1 * 4.8
```

```
MSE=FctMSE(c(GuessBeta1,GuessBeta2), data=DataMockup)
```

```
cat("The MSE for beta1=",GuessBeta1, "and beta2=",GuessBeta2, "is:", MSE)
```

```
The MSE for beta1= 0 and beta2= 83.2 is: 104.96
```

Trial and Error with Grid Search (aka Brute Force)

Guesses for each β :

```
VecBeta1Values=seq(3, 5, length.out=21)
VecBeta2Values=seq(50, 70, length.out=21)

data.frame(Beta1Values=VecBeta1Values,Beta2Values=VecBeta2Values)
```

	Beta1Values	Beta2Values
1	3.0	50
2	3.1	51
3	3.2	52
4	3.3	53
5	3.4	54
6	3.5	55
7	3.6	56
8	3.7	57
9	3.8	58
10	3.9	59
11	4.0	60
12	4.1	61
13	4.2	62
14	4.3	63
15	4.4	64
16	4.5	65
17	4.6	66
18	4.7	67
19	4.8	68
20	4.9	69
21	5.0	70

Trial and Error with Optimizer (heuristic algorithm, not Brute Force)

```
StartForBeta1Beta2=c(0,83) # 83 is the mean grade
```

```
BestBetas=optim(StartForBeta1Beta2, FctMSE, data=DataMockup)
```

```
cat("Optimized values for Beta1 and Beta2:", BestBetas$par)
```

```
# Static fallback for non-HTML formats (pptx): {webr} live cells  
only run in the browser
```

```
StartForBeta1Beta2=c(0,83) # 83 is the mean grade
```

```
BestBetas=optim(StartForBeta1Beta2, FctMSE, data=DataMockup)
```

```
cat("Optimized values for Beta1 and Beta2:", BestBetas$par)
```

```
Optimized values for Beta1 and Beta2: 3.962827 64.17802
```

MULTIVARIATE OLS WITH A REAL WORLD DATASET

Multivariate OLS with a Real World Dataset

Data

- King County House Sale dataset (Kaggle 2015). House sales prices from May 2014 to May 2015 for King County in Washington State.
- Several predictor variables.
- We will use all 21,613 observations.

Multivariate Analysis – Three Predictor Variables

Sqft: Living square footage of the house

Grade: Indicates the condition of houses (1 (worst) to 13 (best))

Waterfront: Is house located at the waterfront (*yes* or *no*)

Unfitted Model

$$Price = \beta_1 Sqft + \beta_2 Grade + \beta_3 Waterfront_{yes} + \beta_4$$

Multivariate Real World Dataset – Splitting

```
set.seed(777)
Split7030=initial_split(0.7,data=DataHousing, strata = Price, breaks = 5)
DataTrain=training(Split7030)
DataTest=testing(Split7030)
```

DataTrain

	Price	Sqft	Grade	Waterfront
1	221900	1180	7	no
2	180000	770	6	no
3	189000	1200	7	no
4	230000	1250	7	no
5	252700	1070	7	no
6	240000	1220	7	no

DataTest

	Price	Sqft	Grade	Waterfront
1	1230000	5420	11	no
2	257500	1715	7	no
3	291850	1060	7	no
4	229500	1780	7	no
5	530000	1810	7	no
6	650000	2950	9	no

Dummy and One-Hot Encoding

One-Hot Encoding

```
# A tibble: 4 × 2
  Waterfront_yes Waterfront_no
      <dbl>         <dbl>
1         0         1
2         0         1
3         1         0
4         0         1
```

One-hot encoding is easier to interpret but causes problems in OLS (dummy trap) because one variable is redundant. We can calculate one variable from the other (*perfect multicollinearity*):

$$Waterfront_{yes} = 1 - Waterfront_{no}$$

Dummy and One-Hot Encoding

Dummy Coding

We use one variable less than we have categories. Waterfront has two categories. Therefore, we use one variable (e.g., Waterfront_yes):

Dummy Encoding Example

```
# A tibble: 4 × 1
  Waterfront_yes
      <dbl>
1             0
2             0
3             1
4             0
```

Note: dummy encoding can be done with `step_dummy()` in a tidymodels recipe.

Multivariate Analysis – Building the Recipe

```
RecipeHouses=recipe(Price ~ ., data=DataTrain) |>  
  step_dummy(Waterfront)
```

Here is how the recipe later on (in the workflow) transforms the data:

```
# A tibble: 6 × 4
```

	Sqft	Grade	Price	Waterfront_yes
	<int>	<int>	<dbl>	<dbl>
1	1180	7	221900	0
2	770	6	180000	0
3	1200	7	189000	0
4	1250	7	230000	0
5	1070	7	252700	0
6	1220	7	240000	0

Multivariate Analysis – Building the Model Design

Unfitted Model:

```
ModelDesignHouses=linear_reg() |>  
  set_engine("lm") |>  
  set_mode("regression")  
print(ModelDesignHouses)
```

```
Linear Regression Model Specification  
(regression)
```

```
Computational engine: lm
```

Multivariate Analysis – Creating Workflow & Fitting to the Training Data

```
WFModelHouses = workflow() |>
  add_recipe(RecipeHouses) |>
  add_model(ModelDesignHouses) |>
  fit(DataTrain)
print(WFModelHouses)
== Workflow [trained] ==
Preprocessor: Recipe
Model: linear_reg()

— Preprocessor —
1 Recipe Step

• step_dummy()

— Model —

Call:
stats::lm(formula = ..y ~ ., data = data)

Coefficients:
  (Intercept)          Sqft          Grade Waterfront_yes
    -570056.2         179.6       95214.1       868338.1

glance(WFModelHouses)
# A tibble: 1 × 12
  r.squared adj.r.squared  sigma statistic p.value    df logLik   AIC   BIC
  <dbl>      <dbl>      <dbl>    <dbl>    <dbl>   <dbl> <dbl> <dbl>
1    0.581    0.581 238574.    7002.      0       3 -208785. 4.18e5 4.18e5
# i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

Multivariate Analysis – Predicting Testing Data and Metrics

```
DataTestWithPredictions =  
augment(WFModelHouses, new_data=DataTest)  
metrics(DataTestWithPredictions, truth=Price,  
estimate=.pred)
```

```
# A tibble: 3 × 3
```

	.metric	.estimator	.estimate
	<chr>	<chr>	<dbl>
1	rmse	standard	244656.
2	rsq	standard	0.549
3	mae	standard	163358.